

Final Report Cover Sheet

Name: __ Teadora Gildengers _____ __ Theodora Kwok _____

Email: __ tgildeng@andrew.cmu.edu _____ __ tkwok@andrew.cmu.edu _____

Results

In TTLH corner

Delay	Delay (ps)	Delay (FO4)	Notes
Fanout of 4 inverter	38	1	Given by course staff
Read cycle time	1.26n		Time from clock=0 to data out=95%VDD (extracted), includes final latching, on TA test vector
Write cycle time	704	18.5	Time from clock=0 to internal node flip=95%VDD (extracted), TA test vector
Read & Add cycle time			

Power	Power (mW)	Notes
Read total	1.606	On a single read, extracted, TA test vector
Write total	1.041	On a single write, extracted, TA test vector
50/50 Read Write mix	1.636	TA power eqn and test vector (extracted works)
Static	38.32uW	TA power eqn with no vector file
Read & Add total		

Area	Area (μm^2)	Area ($\text{\AA}^2$)	Notes
Total (bounding box)	6373.44	12.59M	
SRAM cell (single, repeat dist.)	0.7348	1451.45	
SRAM array total	3712.145	7.332M	Measured bounding box of sram cell array
Cell area efficiency (%)	58%		Cell area / total
Adder			

SRAM Cell	Value	Notes
SRAM cell read SNM	.425V ²	See diagram in appendix
SRAM cell read current	2.04e-8 A	Average current from VDD into cell over 1 read
SRAM cell BL leakage	5.7e-8 A	When wordline is off, but bitline is switching.

Section 1: Architecture and Floorplan

In this section, you should describe the basic architecture of your design and why you chose the architecture that you did. Describe any novel aspects of your architecture. How did you partition the design? How did you decide to physically arrange the blocks? What clocking style did you use in the design? If you used any clocks besides clk and clk_b, how would you generate those clocks? How did you decide on a decoder partitioning? How did you decide on the data input and output muxing? Etc.

We used the standard 6T cell with modern cell layout. We use a 64x64 size cell array with 4:1 column muxing to have an aspect ratio closer to square, and shared nwells as much as possible by flipping the design.

Since the whole design is ~130um wide, the word line wire delay from one end to the other is significant. We picked a T style topology when placing the row decoder and predecoder to equalize the delay on both sides. For the row decoder block, we used two 3:8 predecoders. The height of the individual row decoders is an integer multiple of the cell height for layout purposes.

We used the provided clocks in the design, with the exception of the bitline reset. Bitline reset runs on a pulse generator derived from the provided clock. We did this so we wouldn't have to distribute bitline reset signals across the height of the array. We decided to put one pulse generator per four bitline reset blocks, so that the pulse generator does not have to be large.

Section 2: Circuit Design

In this section, you should describe the types of circuits you used in your design, why you chose them, and any issues that arose due to your choices. How did you size the devices in the circuits? Show any hand analysis (e.g. logical effort) that you did. Describe any simulation experiments you did to help size your circuits.

We used a Nambu OR3 gate at the start of the predecoder, followed by a tristate with an enable signal coming from the signal generator. The NambuOR gate can fire faster because of its dynamic precharge and the tri-state with enable signal ensures that the wordline doesn't fire on a NOP. We chose to use 2 3:8 predecoders to minimize the amount of logic that had to be performed in the row decoder so we could save space. Generally speaking the devices in the circuit were sized to achieve a stage effort of about 4, and always below 6. When sizing the row decoder, I limited the size of the final inverter to a lower fanout because I didn't want to take the area of a 55-70x minsize inverter. To determine the size of the final inverter in the row decoder, I created a testbench I loaded with a row of extracted cells.

Rdec c in	rdec cout	actual fanout	PE	ideal # of stages	SE	FO_inv	FO_nand			
0.36	33.22	92.27777778	180.8644444	3.749382505	3.667231275	3.667231275	3.056026062			
			used 1.4 for NAND LE at 4x minsize							
Sizing										
NAND	INV	Nand	INV							
4x minsize	6x minsize	10x minsize	70x minsize???							
Cin	Cin	Cin								
0.36	1.100169382	4.034575566	12.32976808	37.68009259						
	6.112052124	11.20715435	68.49871156							
NAND	INV	INV	INV	NAND	INV	INV	INV	SE	FO_inv	FO_nand
4x minsize	4x minsize	8x minsize	16x minsize	13x minsize?	51x minsize			1.915001638	1.915001638	1.367858313
Cin										
0.36	0.6894005898	1.320203259	2.528191403	4.84149068	9.271462583	17.75486604	34.00059754			
	3.830003277	7.334462549		13.44858522	51.50812546	98.63814464				
Pdec in	Pdec out	fanout								
0.24	4.12	17.16666667								

Image: screenshot of sizing spreadsheet used on initial version of row decoder.

The row decoder was initially NAND-INV-NAND-INV, with the final gate ANDing in the clock to turn off the row decoder. While this worked initially in schematic, it made our WL pulsewidth too short as we added in extracted views. Teadora redid the decoder to remove the 2nd NAND gate. The row decoder was sized based on the input capacitance of the first NAND gate and the output capacitance of a row of cells. We found the capacitance of a row of cells by extracting a 1x32 layout and sweeping the capacitance until the delay was equalized, then doubling the capacitance value. We laid out 1x32 because the wordline is split by the row decoder in the center.

We also sized the loads on the output inverters of the latches to achieve roughly a fanout of 4. Our final column mux predecoder design had 4 latches (a0, ab0, a1, ab1), but had two halves so it could fit on either side of the decoder chunk. We had to reduce the inverter sizing on the colmux predecode NAND gates when we split it, since each latch was loaded twice. We also stepped up to a final colmux predecode transistor size of around 12-16x (see schematic for specifics) to drive all the gates on the colmuxes.

Describe how you generated the control signals for the design. How much extra margining is designed into the replica path? How well did the replica track the real design over corners/variations? What corner simulations did you perform to ensure that the design is robust?

The control signal block outputs Write Enable (we), Sense Enable (se), and Enable (en, en_b). I use a NAND2 gate with latched read/write values to see if there is a NOP (op, op_b) signal. Op and op_b are used to in a NOR3 to fire the we and se signals. The we signal has as little of a buffer as possible so it fires early. The se signal has a larger buffer to fire later. I made sure to step up the signals on the output so they could drive the full gate load + wire capacitance. This was done with a rough calculation based on the size and number of gates. I also step up the enable signal from the op signal so it can drive the predecode separately from the we/se logic. The bitline reset is separated from the control gen. We use a pulse generator to assert bitline reset briefly at the beginning of each cycle. By connecting it to the clock distribution network we don't have to route across the height of the array, reducing wiring constraints and timing delay.

Describe how you took into account interconnect capacitance and resistance. Where were the long wires in your design? What metal layer, wire width, and wire spacing did you use for these wires? How did you model these wires in your schematics/simulations?

The long wires in the design are the wordlines, bitlines, predecode input lines, VDD and VSS rails. We used min width for all wires. VDD and VSS are primarily on metal3, especially in the cell array and row decoder. However there are also some metal2 rails in the cell I/O. We often had to use minimum wire spacing to fit all the connections in layout, but we tried to alternate “loud” wires (decoder inputs, address bits) with “quiet” wires (VDD, VSS) to have shielding.

We used a pi layout for all our wire models. I swept the capacitance until the delay was equalized, then split it into 2 capacitors separated by a resistor. I modeled the resistance of these wires with the square method and by referencing the pdk guidelines.

One example of how wire models were used to size blocks is the testbench I made for the decoder (see images in Appendix). I took 2 predecoders, put a wire model on each output to represent the wire running all the way to the top of the array, and then loaded each output 8 times with the input capacitance of a single row decoder. I put an extracted row decoder on the output and loaded that row decoder with an extracted row of cells. The purpose of this was to analyze every stage in the decoder and determine which parts were contributing the most to delay.

Section 3: Results

Please be sure to note what process corner you are simulating in.

A) Performance

What performance did your design achieve? What test vectors did you use to stress the critical paths? What was the delay breakdown of the design (e.g. address/data input à WL, WL à bitline, bitline à data out). What was the critical path in the design? How did you determine this? Was this what you expected to be the critical path?

Our design passed the 422 testbench. We tested the critical path of the design by reading and writing to the 255th wordline (63rd wordline with column muxing). We wrote opposite value test vectors (1010 ...) then (0101...) to stress the nodes as much as possible. See test vectors in appendix. The delay breakdown was 397ps from clock to WL enable, 341ps from WL enable to WL high, 71ps from WL high to BL pulled down (on a write). The critical path in the design was through the decoder (firing the highest WL), then the sense enable signal and final output latch.

B) Power

What was the power dissipation of your design (static and dynamic)? What simulations did you run to determine this? What test vectors did you use? What was the breakdown

of the power of the design (e.g. clock power, static power, sense amp power, decoder power, write driver power, etc.)?

Power dissipation at TTLH was within spec for schematic. We measured this with the provided TA equation and provided checkoff vector.

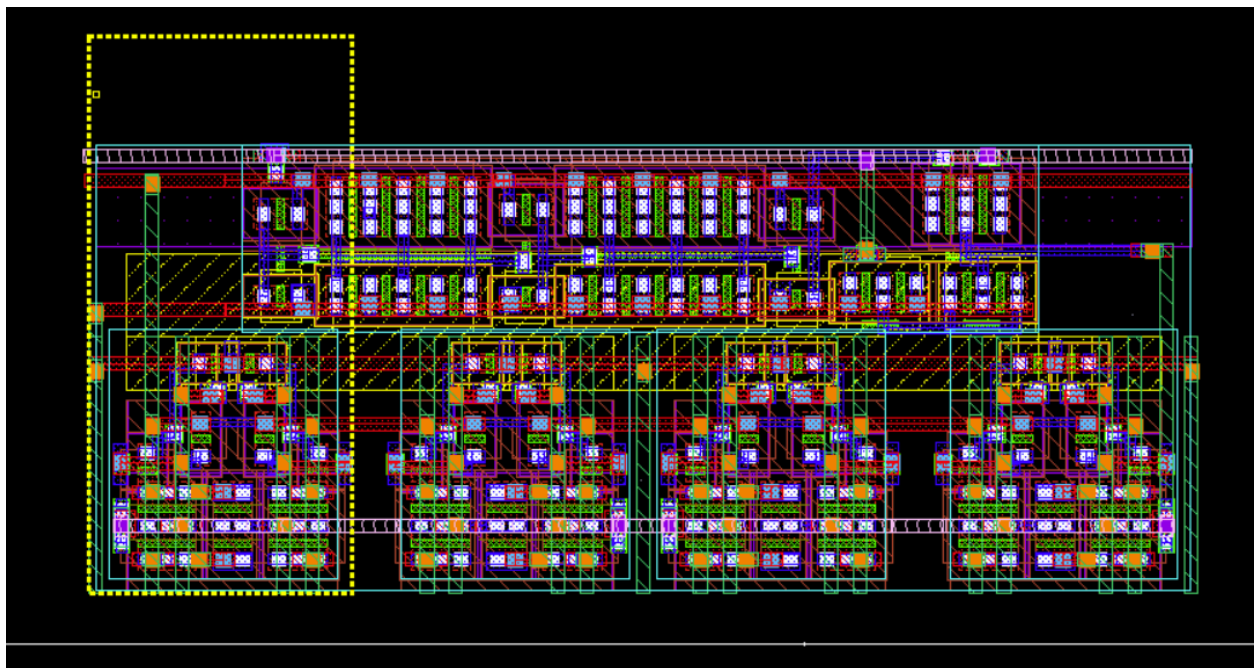
C) Area

What was the area of your design? How did you determine that area? What was the area efficiency of the memories in the design (i.e. cell area / total area)? For the cells that you did layout please include a printout of the layout. Please include a floorplan of the design showing the dimensions of each of the major blocks. Note where the long wires in the design are, what metal layers they are on, and their width and spacing.

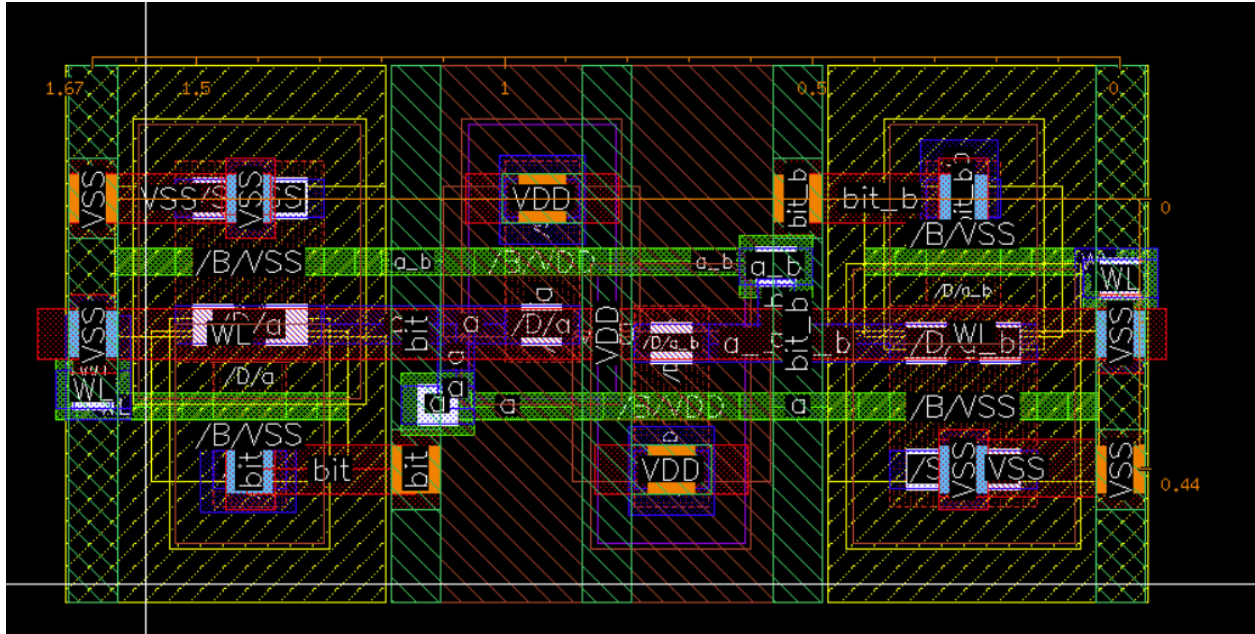
Area is in spec and is on the report sheet. Total area was measured by the bounding box. Cell array area was measured by the bounding box of our cell array block.

Layout screenshots:

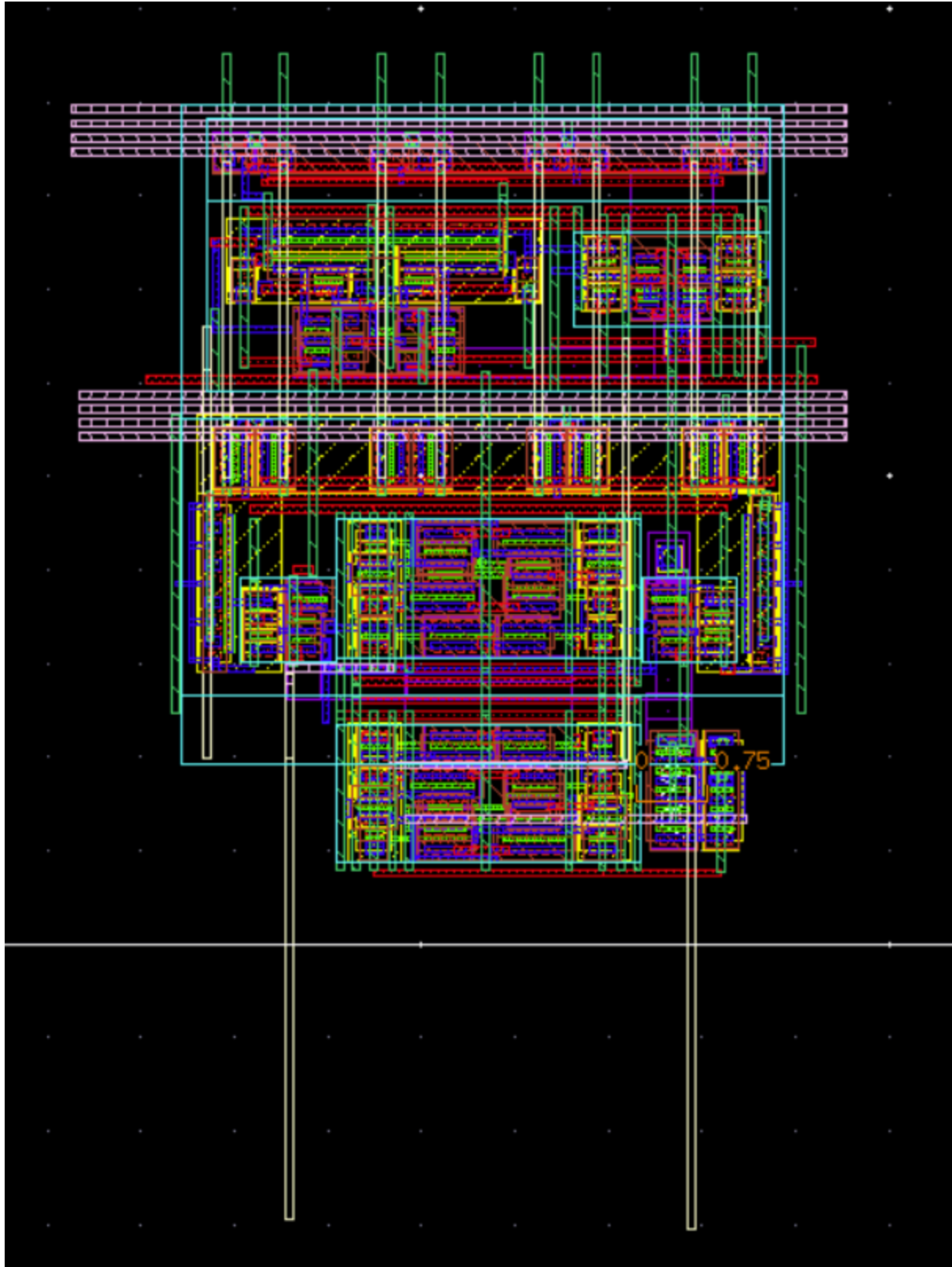
BL reset for 4 columns:



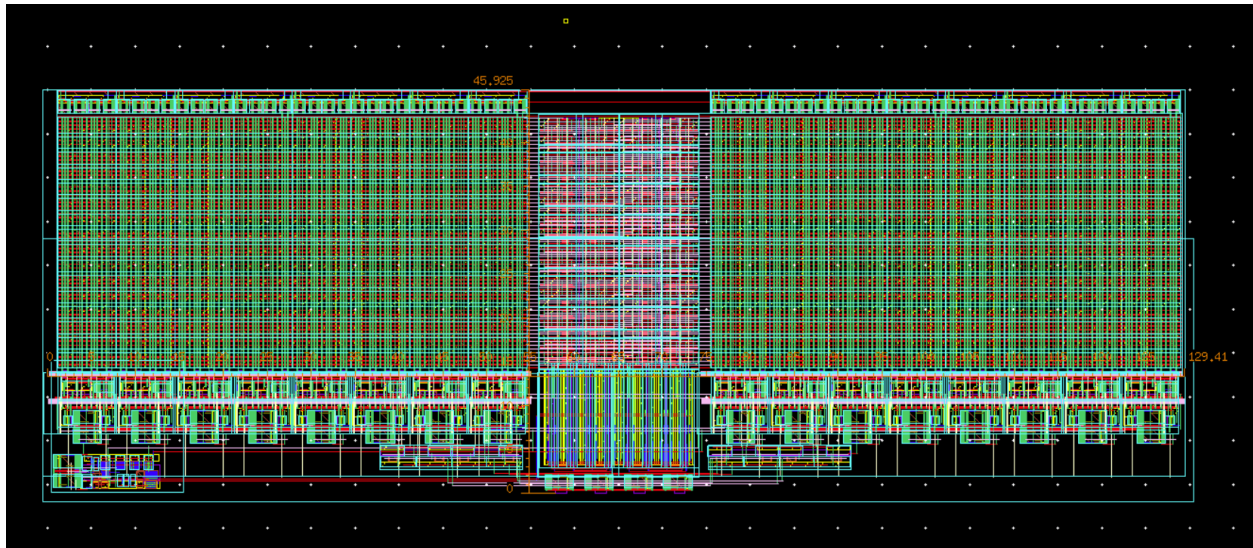
Single cell with measurements:



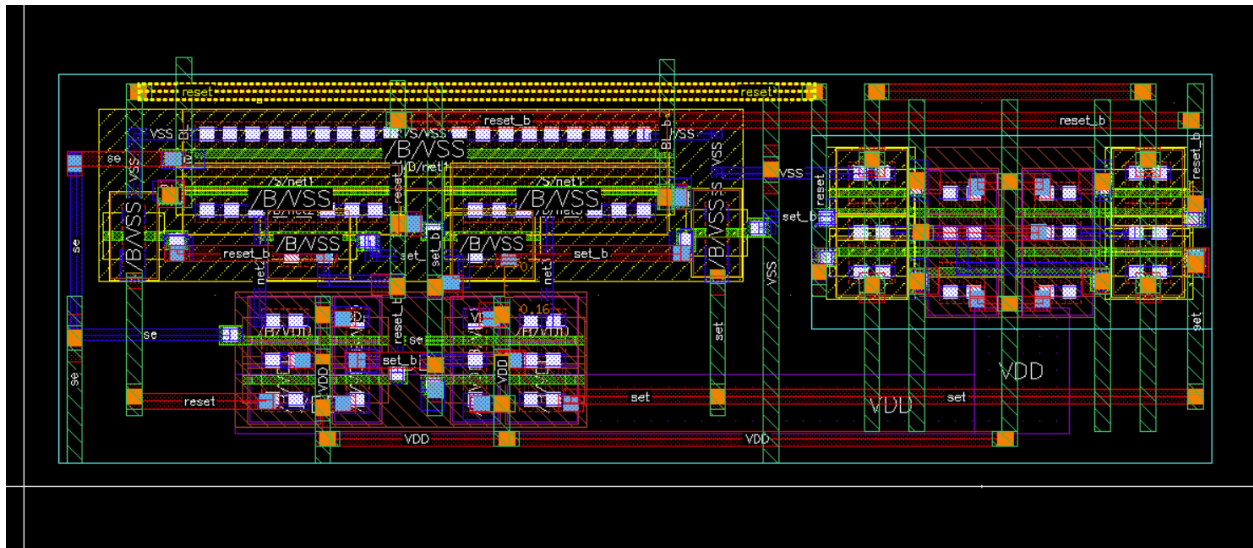
Read Write driver for 4 columns:



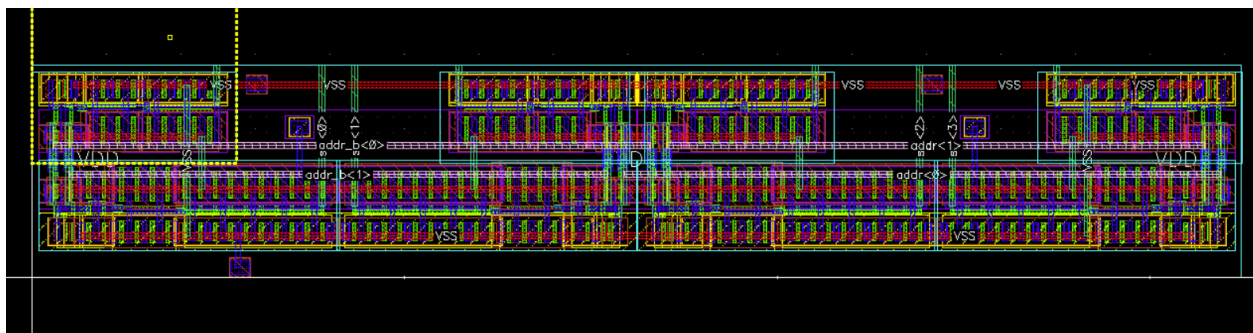
Full SRAM layout:



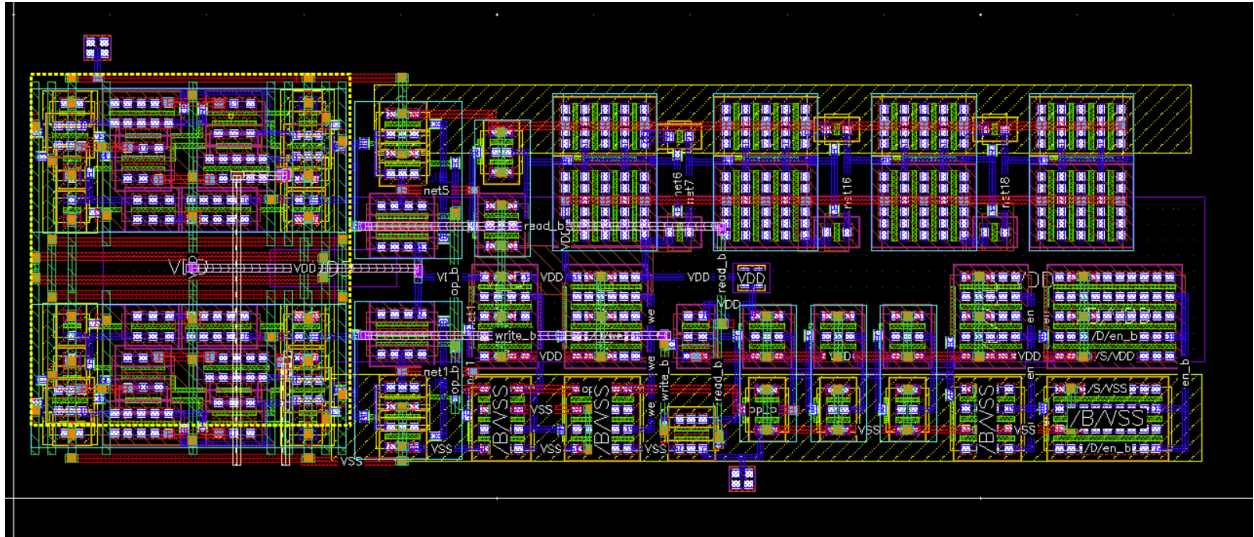
Strongarm SA:



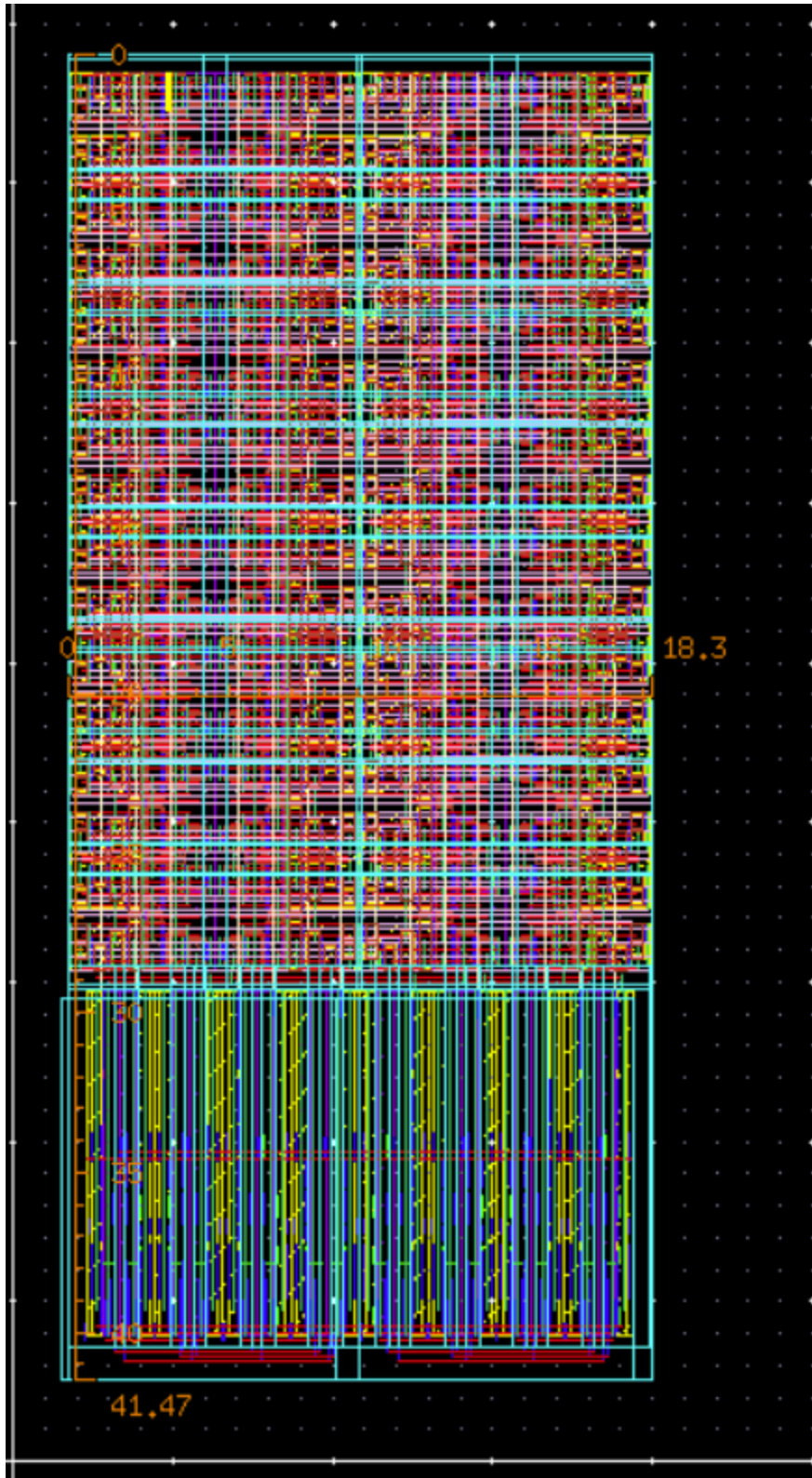
Column predecode:



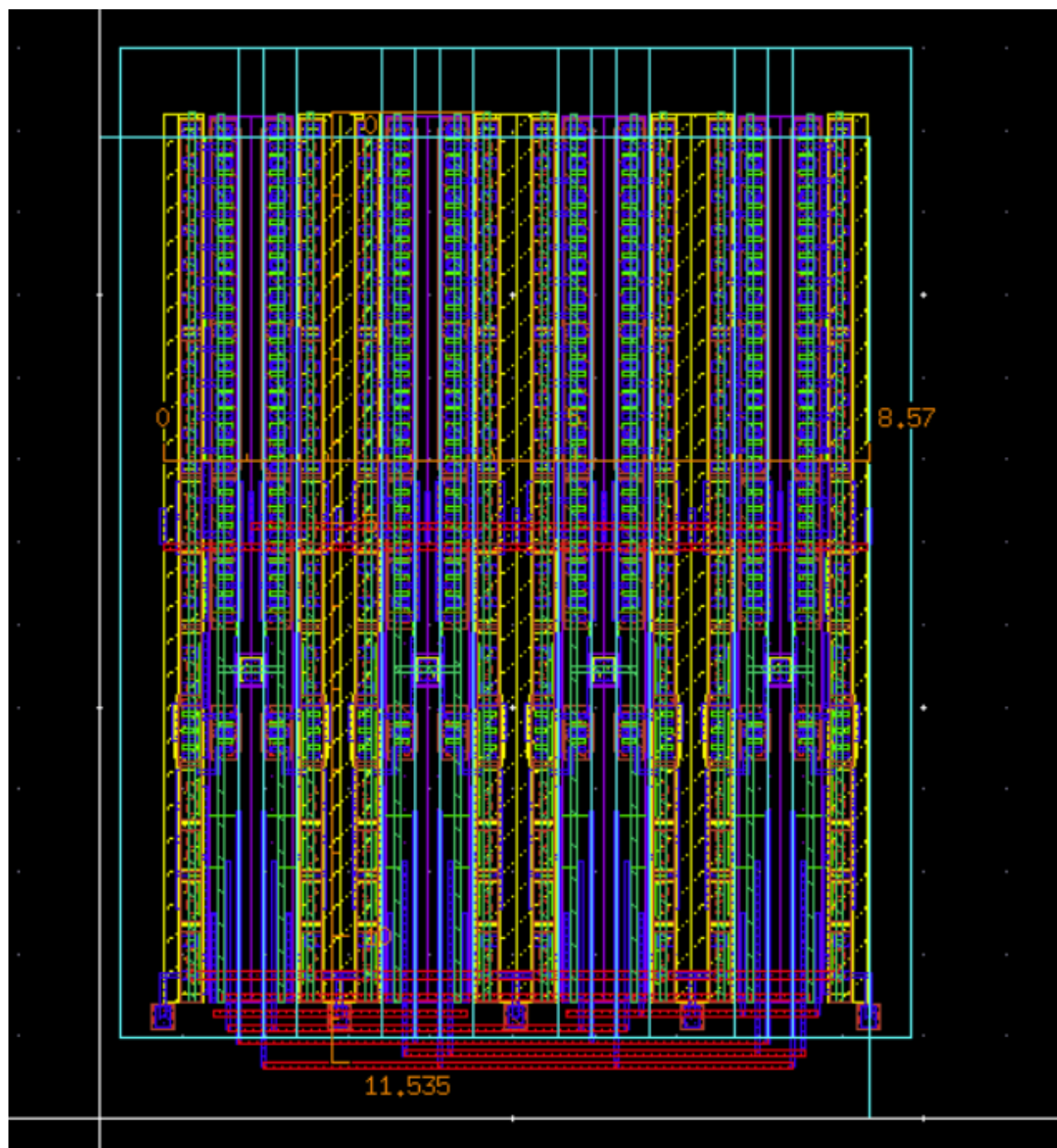
Control gen layout:



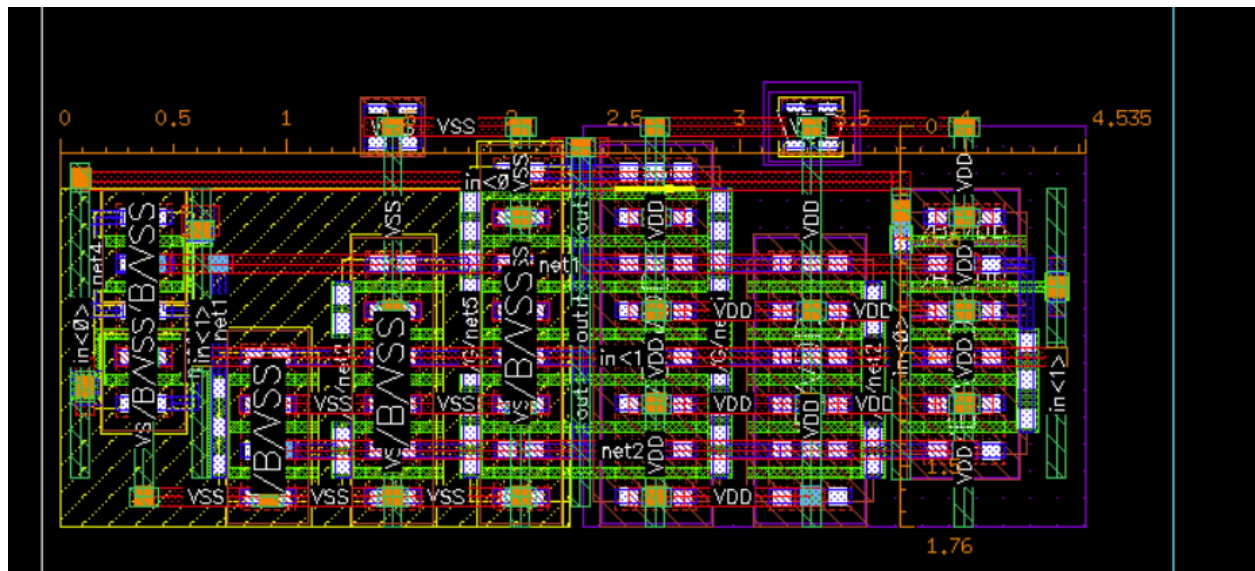
Decoder block:



Predecode 3:8 layout:



Row decoder layout:



Section 4: Testing

What was your testing methodology? How did you identify the critical path of your design? What test vectors did you use? Describe your test infrastructure. How did you generate the clock and inputs? What testing did you do to ensure that your design will work under all process corners and worst-case device mismatch? How did you take into account the clock uncertainty? How did you ensure that your gates were robust? What special schematics/simulations did you run to ensure the robustness of your design?

Our testing methodology was to test everything in schematic and then layout. We did this first for individual blocks, then larger chunks of the design. We moved to the TA testbench as quickly as possible to confirm full design correctness in schematic. We generated the clock with a fanout of 3 clock block to ensure accurate waveforms. Inputs were generated by vbit with shaping to match the testbench (sharp edges for address bits, shaped waveforms for se, we, etc). Our default testing corner was TTLH and we didn't test in other corners until too late. We didn't have to worry about clock uncertainty since that wasn't part of the design spec. We ensured that our cell was robust by running an SNM sweep at the beginning of the design process. When it came to testing, we created extra worst case test vectors using the python script and ran them at multiple corners. They only passed in TTLH and by that point it was too late to fix things.

Test vectors:

Crit path:

W 255 21845

R 255

W 255 43690

R 255

Write wait read

W 10 25

W 15 0

R 15

W 20 15

R 20

R 10

“Write wait read” variation

W 0 0

W 63 15

W 62 10

W 31 25

R 0

R 31

R 63

R 62

Colmux switch test:

W 10 25

R 10

W 138 4

R 138

W 0 31

W 1 30

R 0

Section 5: Design methodology

Briefly describe your design process and the evolution of your design. What were your key design decisions? Were there any design directions that you explored, but rejected? What design constraints did you use to guide your design? What (if any) tools or scripts did you use to aid the design process? What major issues did you encounter in doing the design?

How did your team divide the design workload? Briefly describe what parts of the project each team member worked on.

Major issues: we experienced key extraction issues that delayed us a week and then had to copy everything from andrew space to ece space. There was poor time management. I wish we had started using the full schematic with TA testbench earlier to get to a correct design faster. I wish we had extracted our designs as early as possible so we incorporated it into our simulations earlier, but we were dealing with extraction issues. We redid the row decoder Thanksgiving weekend. The last week of the project was particularly bad because even when everything was extracted the design still didn't work, so we spent days looking at it bottom up to identify every little bug. We still don't know what the bug was and it could have been timing.

Teadora did the design and layout of the decoder, design of control gen, design of bitline reset, sizing for column predecode, simulations, and writing tests. Theodora did everything else: cell layout, cell array layout, bitline reset layout, read/write driver layout. We both did design integration.

The more tired we got the longer it took us to make progress and the worse decisions we made, but we still had to keep going.

Section 6: Comments

This is the section where you get to provide feedback about the project. Please answer the following

questions, and feel free to add whatever additional comments you would like.

- 1. Did the lectures and problem sets adequately prepare you to do the project?**
- 2. Was the amount of design freedom in the project appropriate?**
- 3. Was the project scope appropriate for the amount of time given?**
- 4. Was the project support adequate?**

I wish we understood from the beginning how bad it was to have extraction issues and that we needed to sit down and fix it immediately, not delay it.

- 5. How would you suggest we improve the project in the future?**

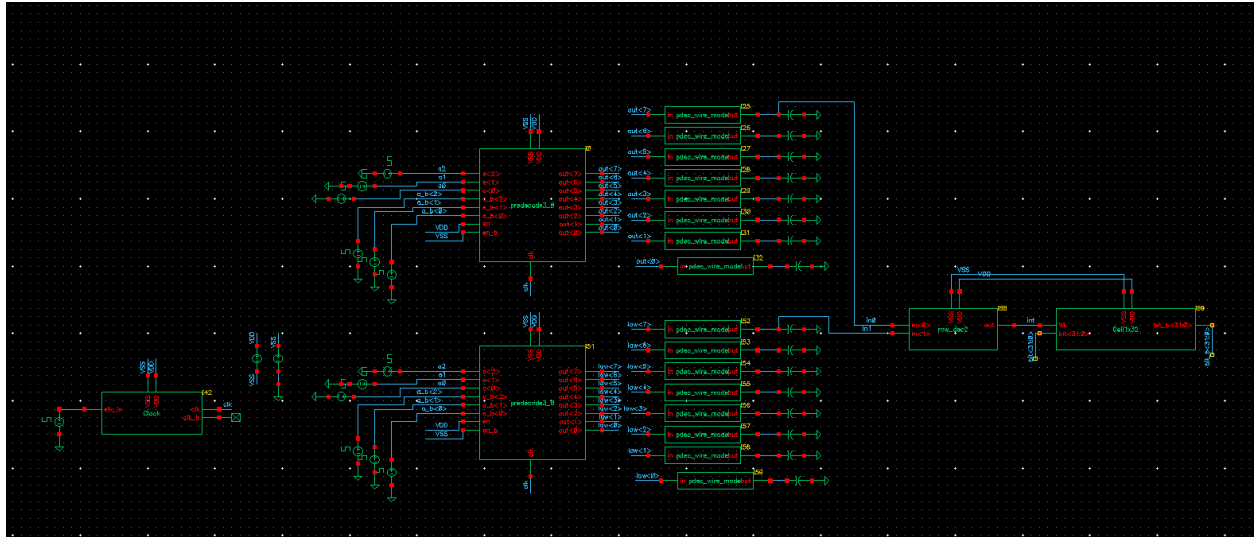
Please don't issue labs in the back half of SRAM please release them before fall break.

Appendices

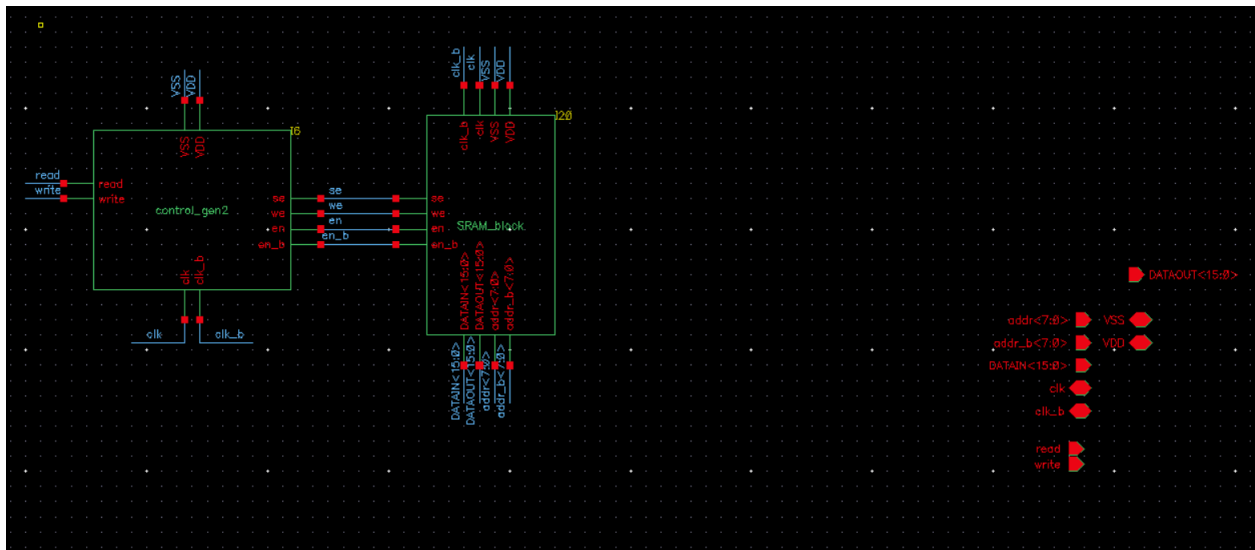
You should use the appendices to give a more complete view of your design. You should include:

B) Schematics – printout of the schematics of your design. You don't need to print out every version of similar gates. Just note that gate X is similar to gate Y, and in what slight ways they differ.

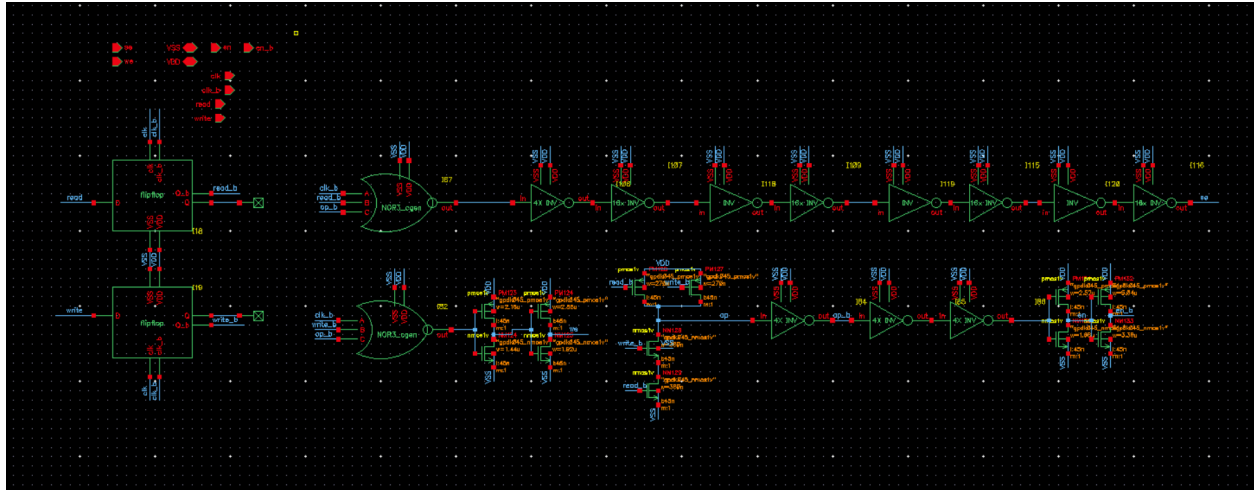
Decoder testing with Wire models



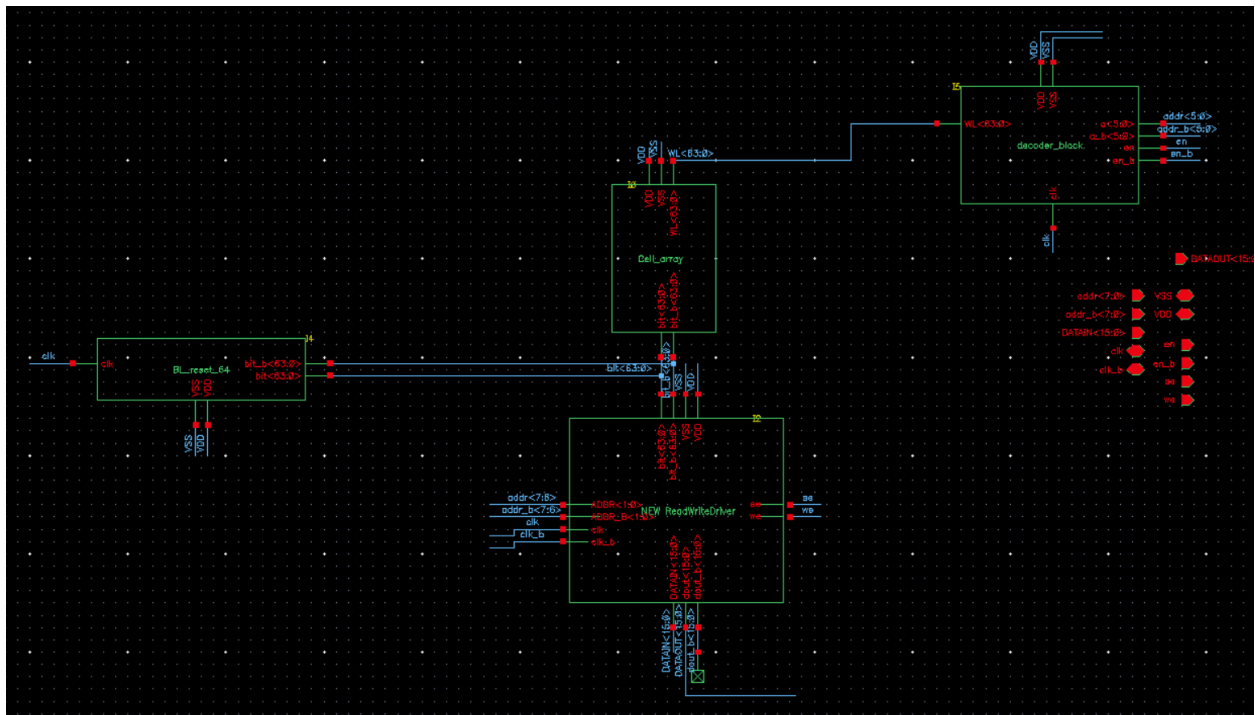
Design screenshots:



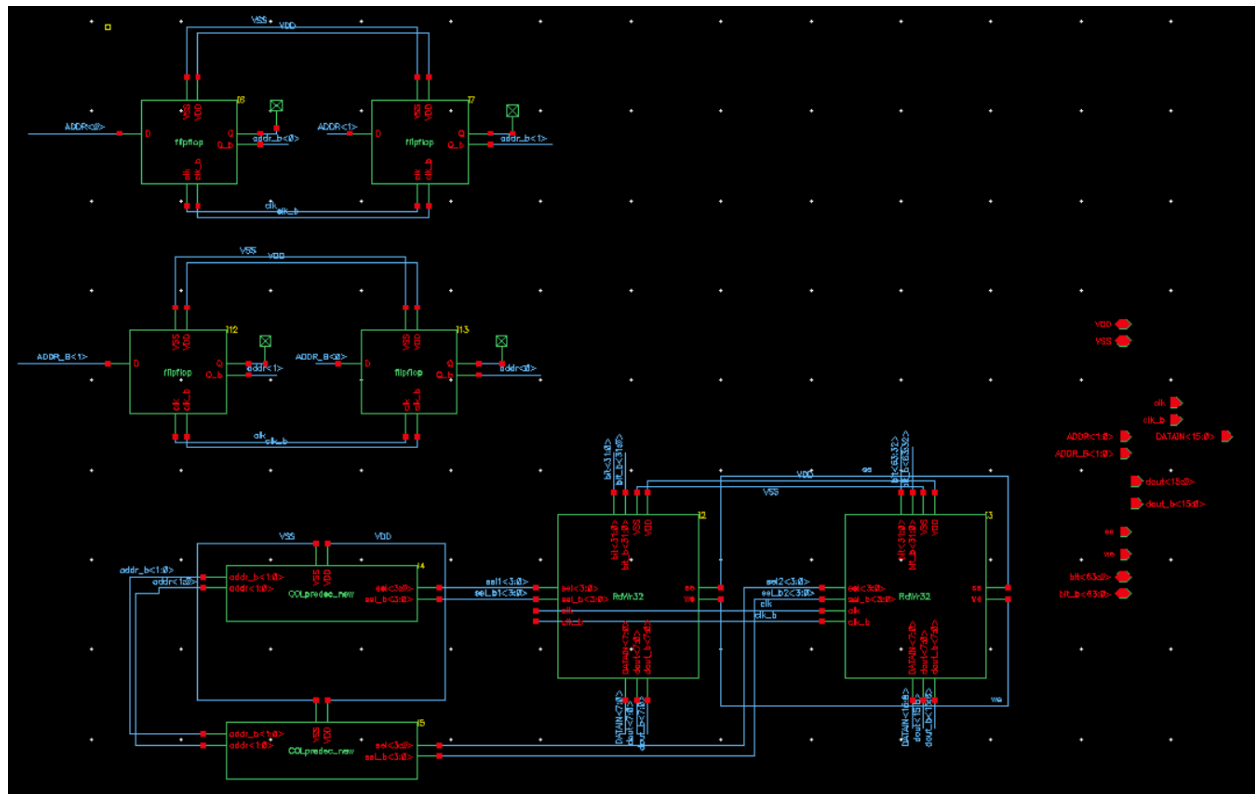
Control gen:



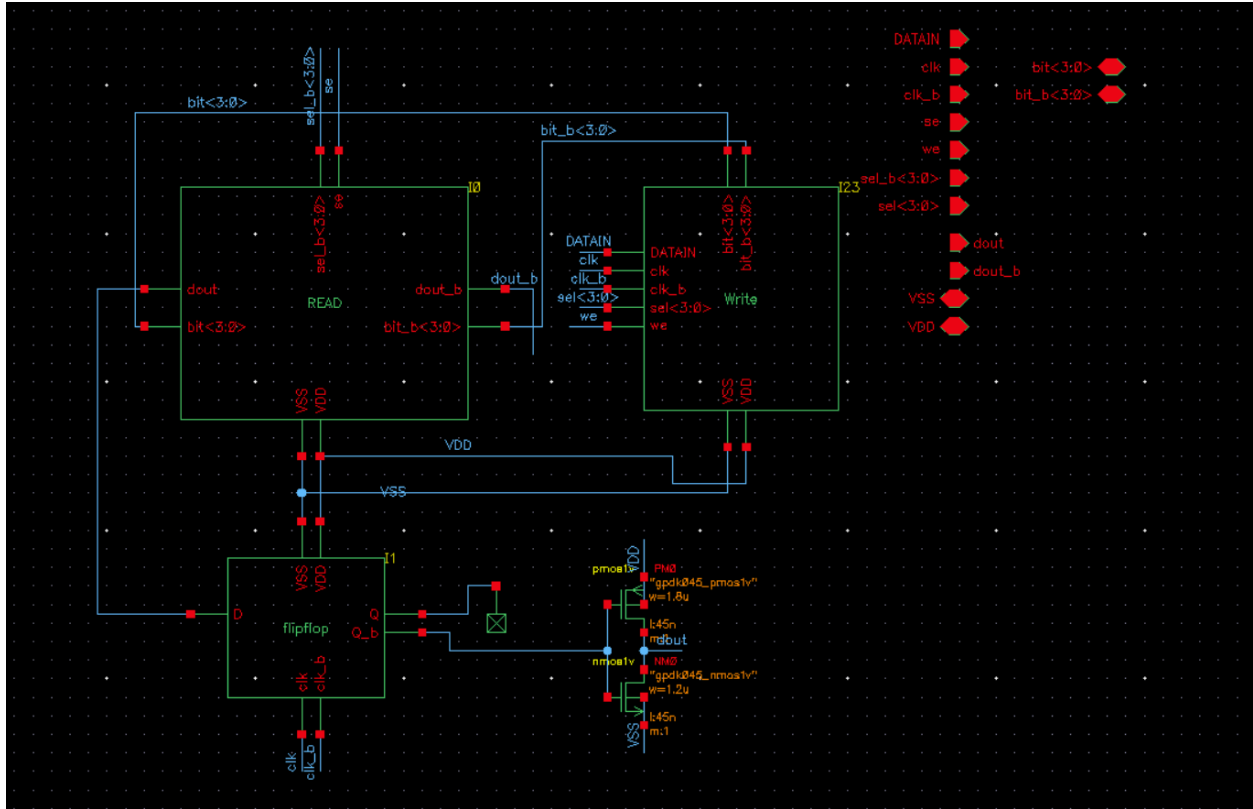
SRAM block:



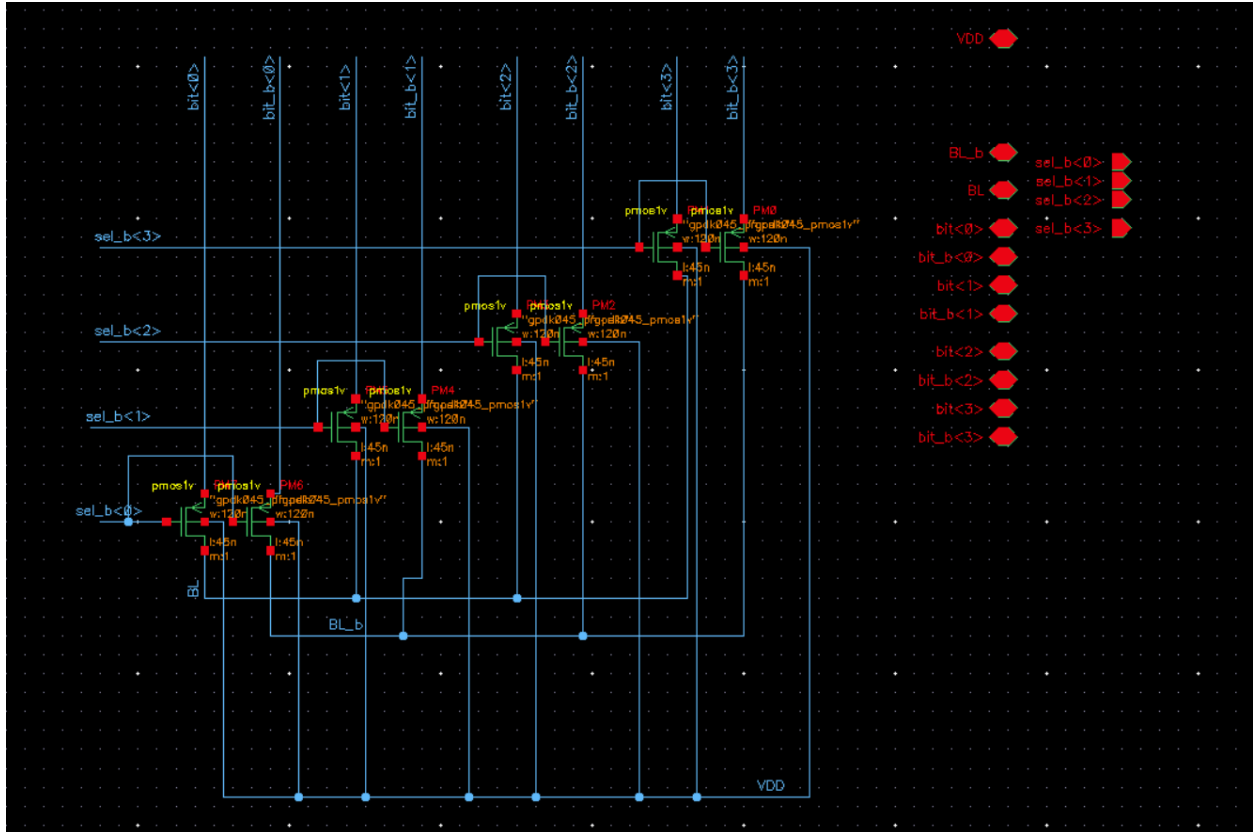
ReadWrite Driver



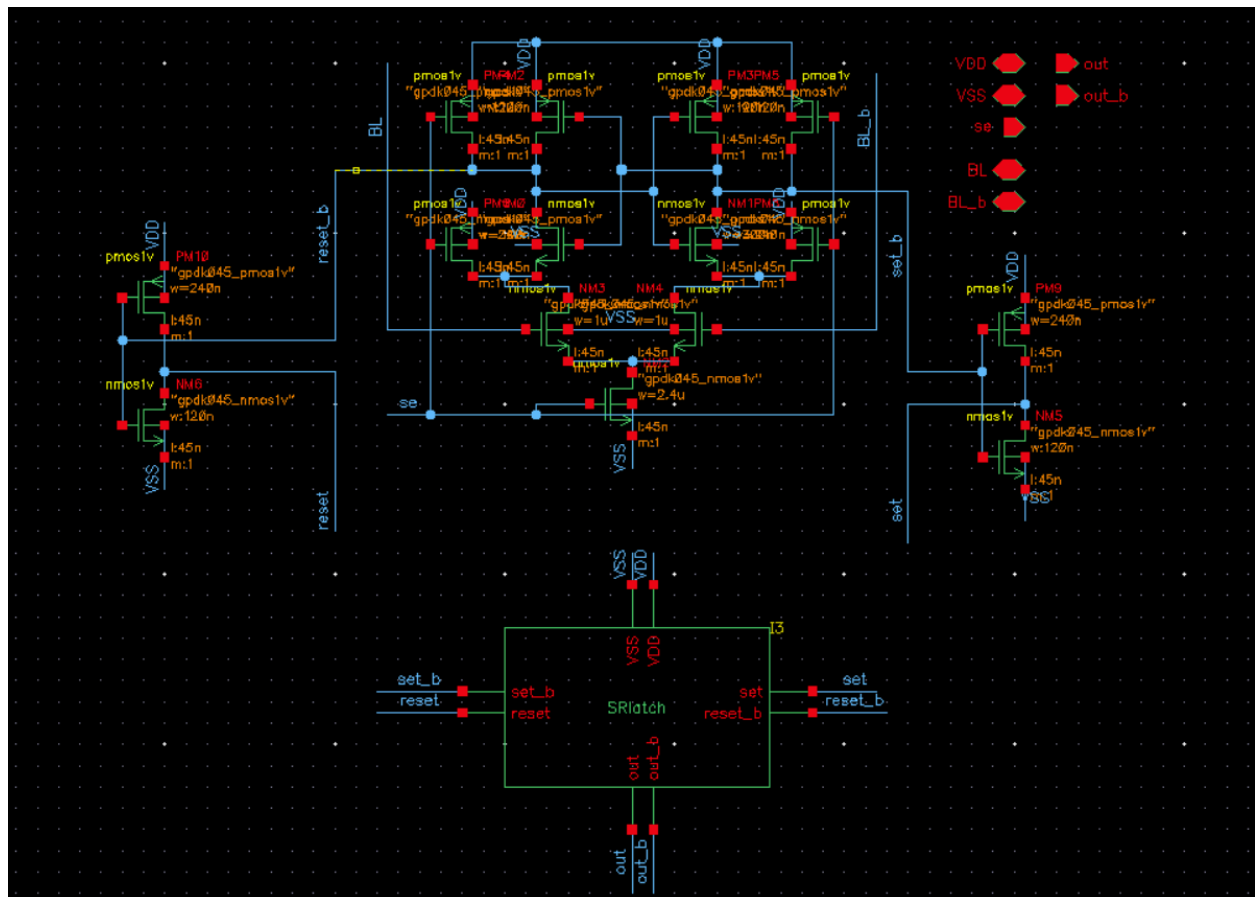
ReadWriteDriver for 4 columns:



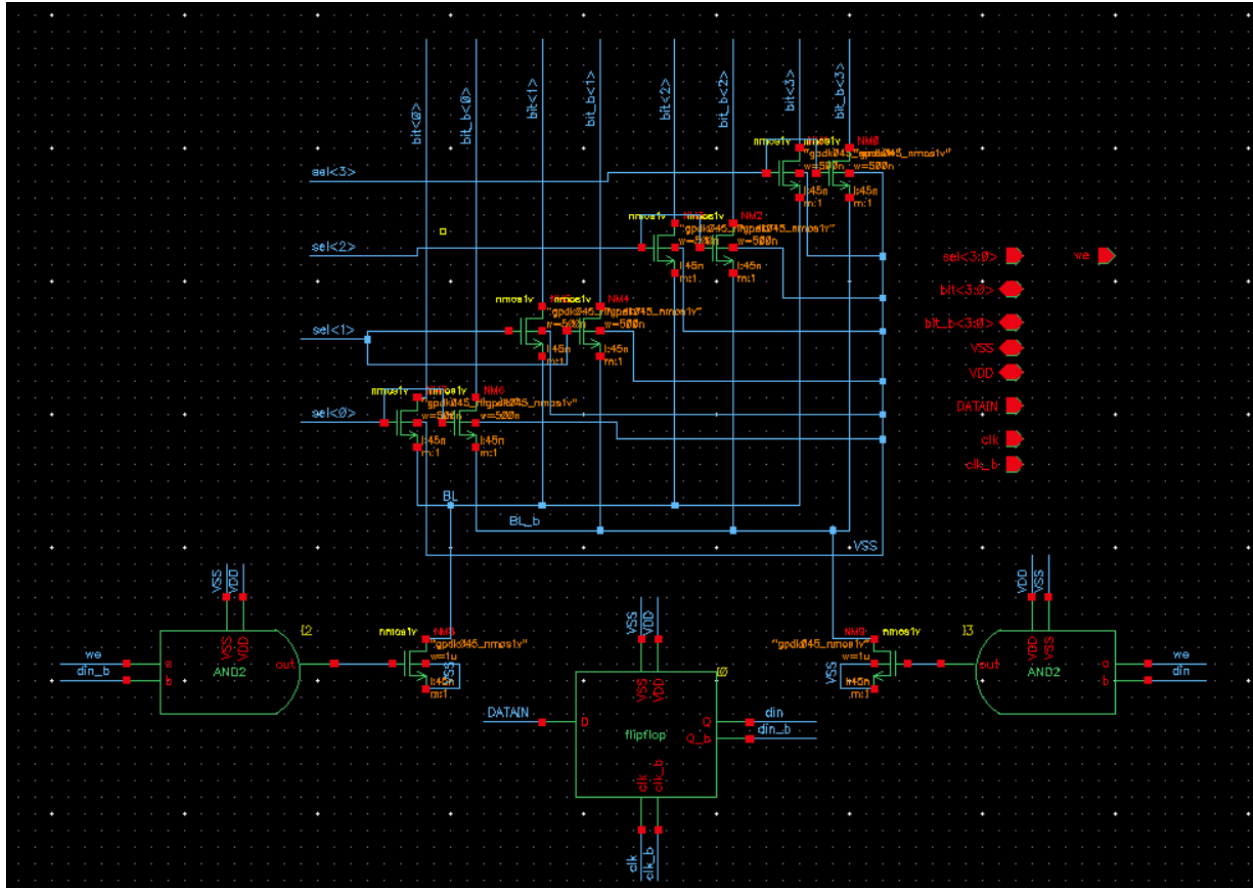
Read colmux



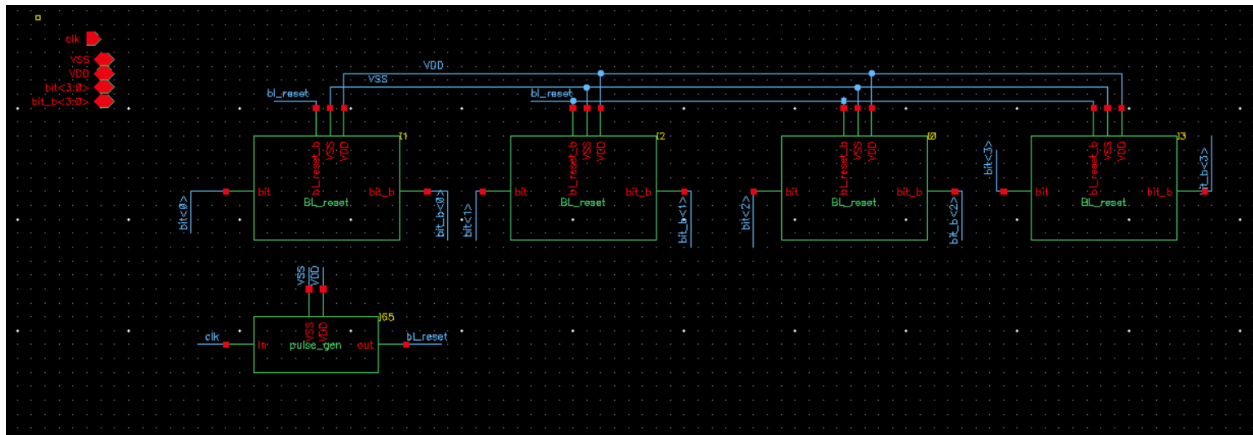
Strongarm sense amp:



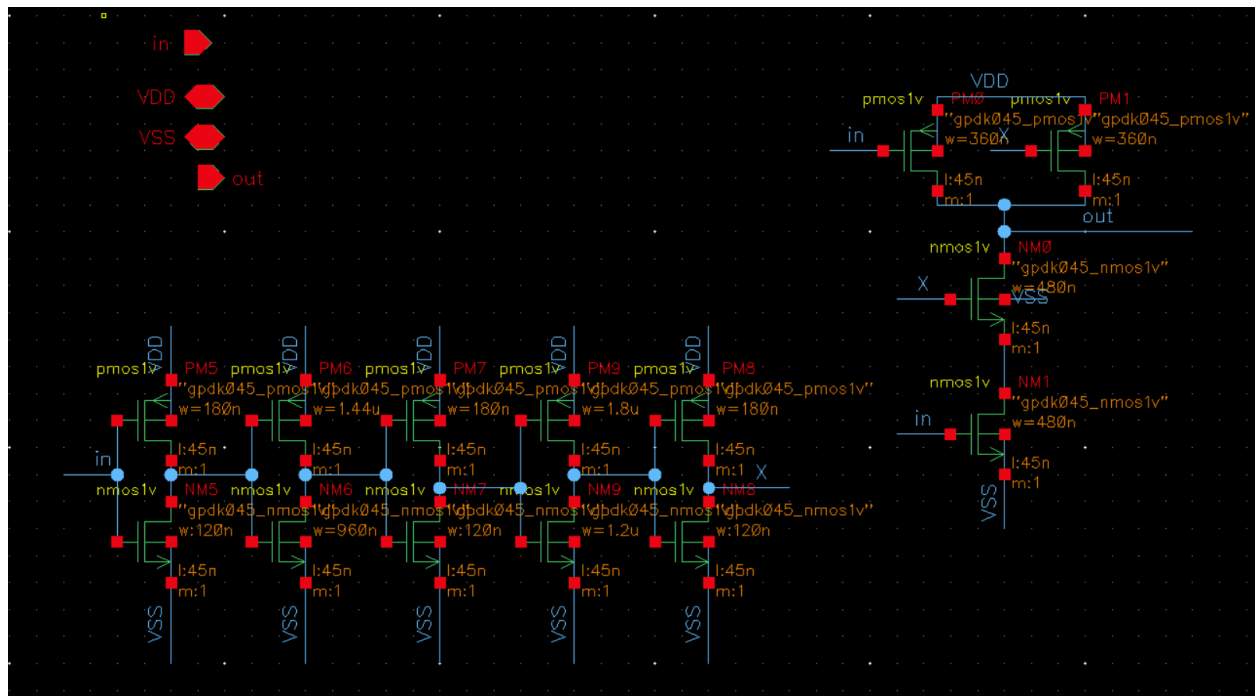
Write driver for 4 columns



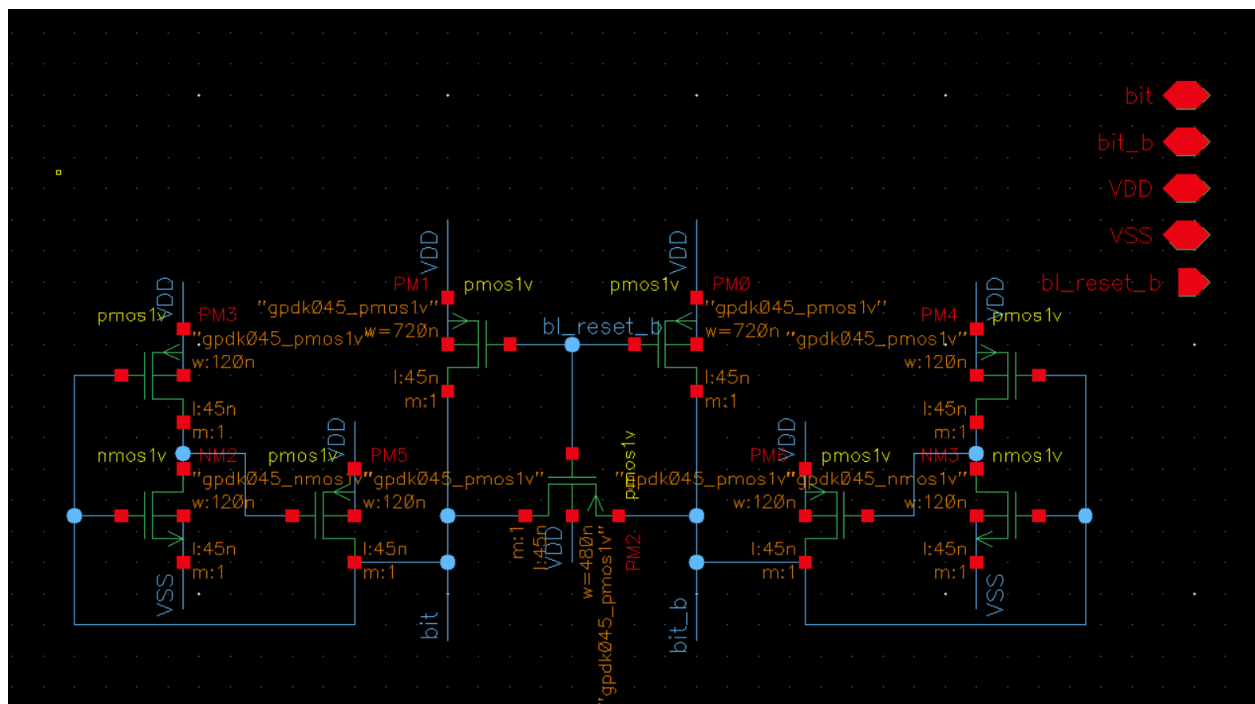
BL reset for 4 columns:



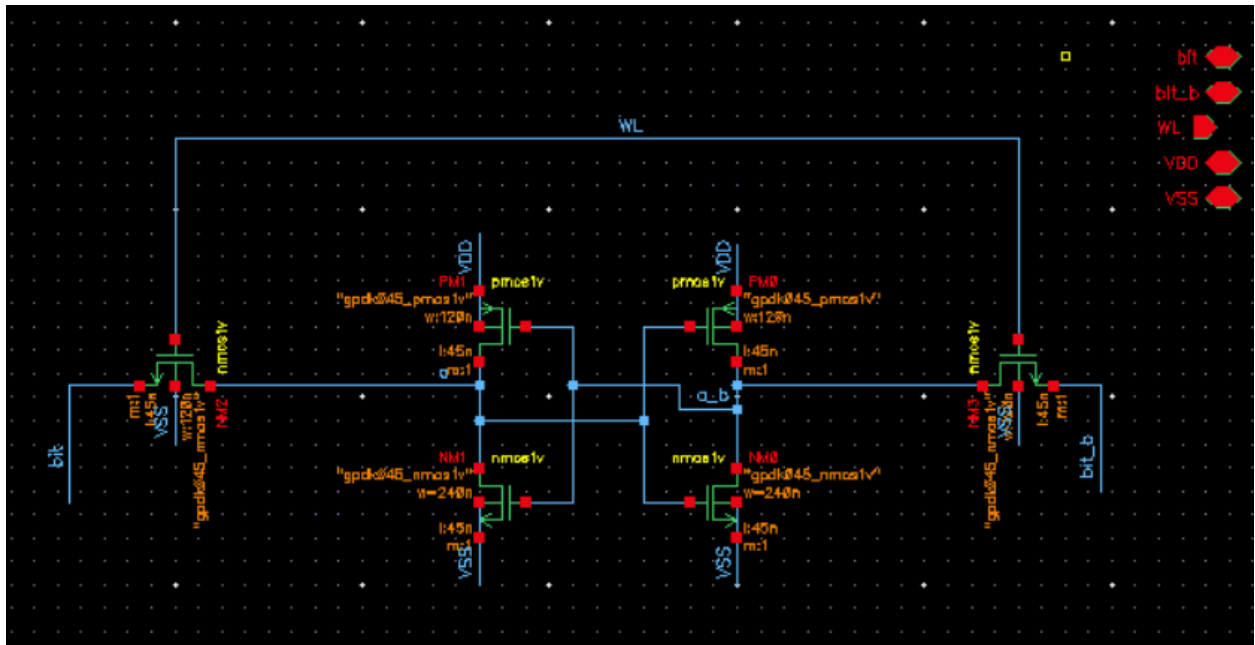
Pulse gen:



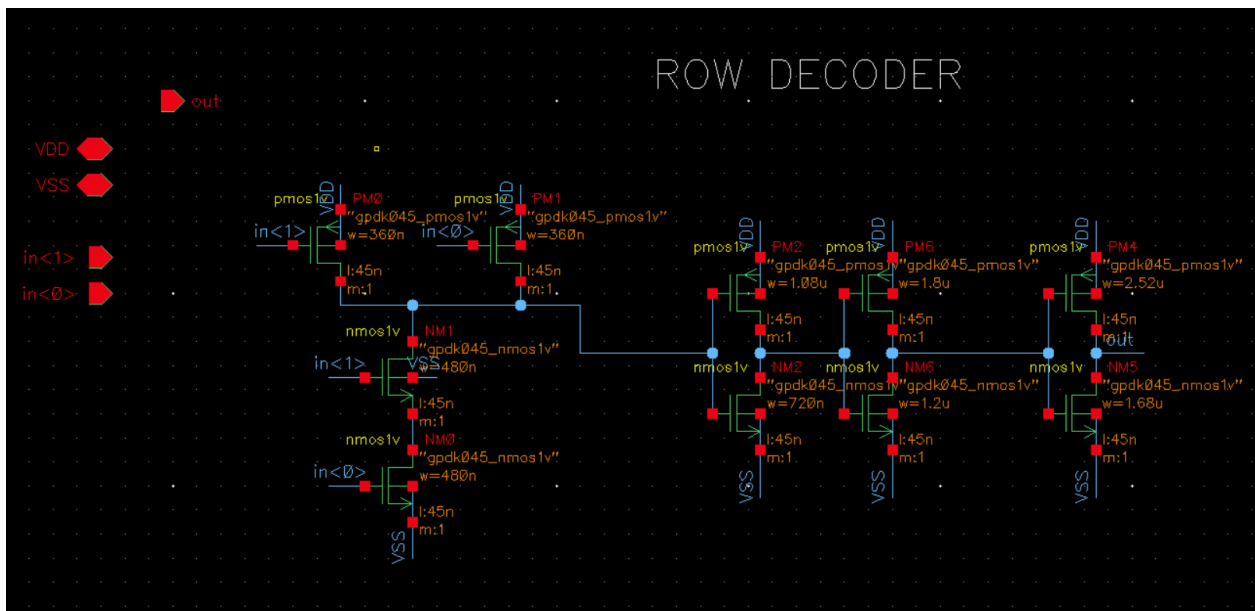
Bitline reset:



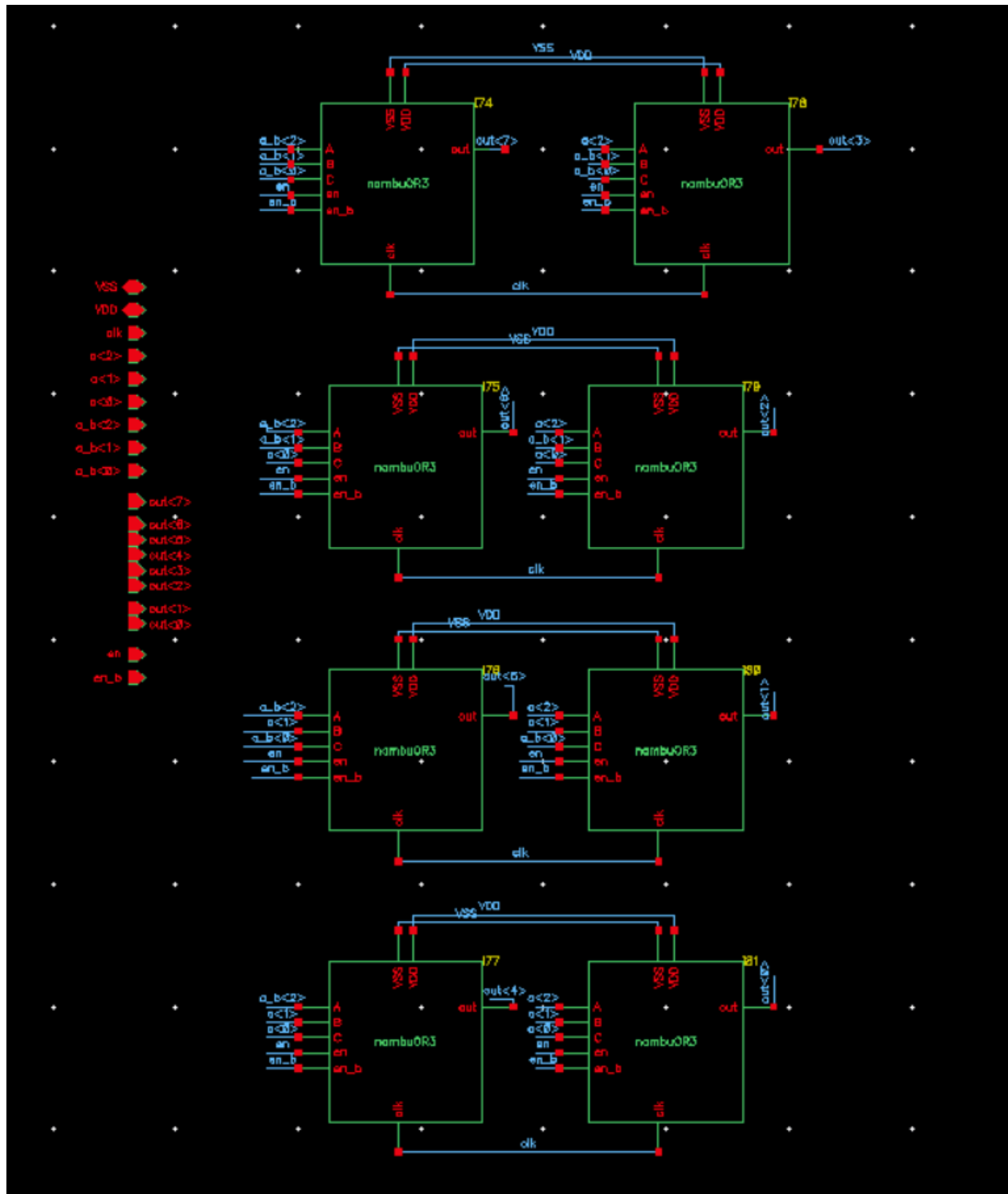
Cell schematic:



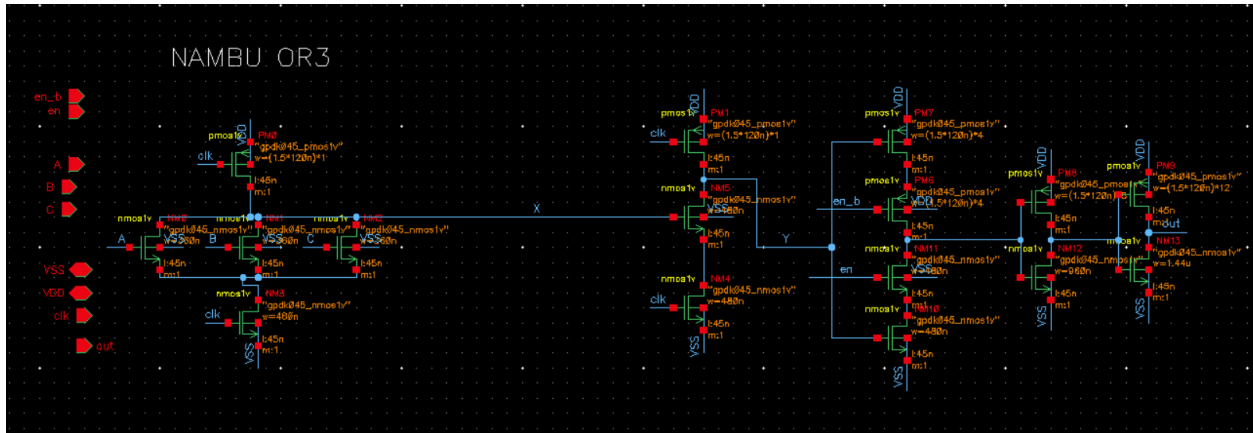
Row decoder (final version)



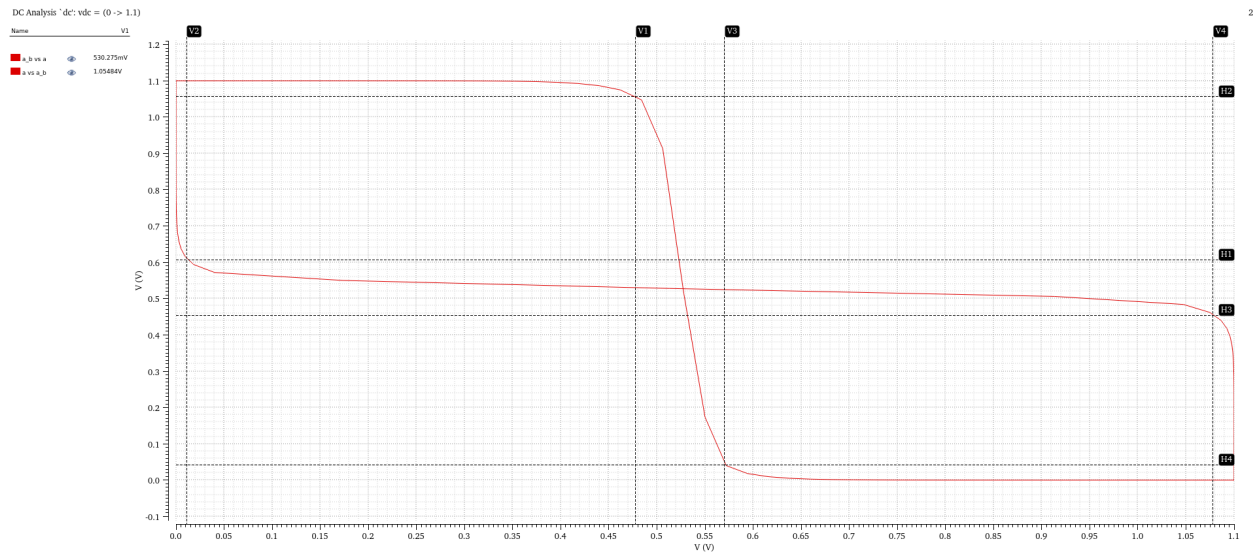
3:8 predecoder:



Nambu or3 gates



C) Waveforms – show your design operating under a worst-case input vector for delay. Show the major intermediate signals. Explain what that vector is and why it exercises the critical path. You can include any other supporting documentation that we will need to evaluate your design.



Extracted snm diagram